University of Arkansas STAT6393V - Week 4

Week 4 - Assignment 4: Uncertainty Quantification (Weather Data)

Group 2: Carlos, Landon, and Elias.

2026-06-18

Introduction

This assignment contains 2 different questions. Question 1 is optional. Question 2 is required, since it is a primer for the final project.

1. Getting UQ based on empirical quantiles The bnn_predict function in the Bayes by backprop section looked like this:

```
bnn_predict <- function(model, x_new, n_sample = 1000) {
    model$train() # set to training mode to get dropout samples
    preds <- torch_stack(lapply(seq_len(n_sample), function(i) {
        model(x_new)$squeeze()
    }), dim = 2) # shape: [n_new, n_sample]

    list(
        mean = preds$mean(dim = 2), # shape: [n_new]
        std = preds$std(dim = 2)
    )
}
```

In its first stage, it returns a tensor whose dimensions are n_test x n_samples. It then takes the mean and SD over the n_samples to return a tensor whose dimensions are n_test x 2. The idea is that $\mu \pm 2\sigma$ contains 95% of all the data.

Instead, we want to get the empirical 95% quantile. Do the following for the noisy sine example:

1. Make the function return a tensor whose dimensions are n_test x n_samples
2. Convert it into a matrix
3. Calculate quantiles for each n_test. One way to do this is via the following command: apply(preds,1,quantile, c(0.05,0.95))
4. Check the proportion of n_test points where these new intervals are wider than the $\mu \pm 2\sigma$ intervals.
5. Submit your code for just the modified prediction function, and how you did your calculations for question #4, and also your result.

2. Getting UQ based on quantiles of the predictive distribution The bnn_predict function in the Bayes by backprop section looked like this:

1. Load up the weather data and subset July and location 1

```
weather <- weather[weather$month == 7 & weather$location == 1,]
```

Keep tmax as Y and pr as X. The remainder of the covariates are the same for the remaining data points, so point in using them.

2. Fit an SPQR model on $Y \sim X$ using MLE. (no need for a train-test split)
3. Simulate a bunch of precipitation values. You can get the range of all precipitation values by using range function, and then generate naybe 10000 values over the range using the runif function. For example, if the range is (0, 32), you can do runit(10000, 0, 32).
4. Put all the pr values in an X_test matrix. Use the predict function to predict the PDF for Y for each X. This matrix would have dimensions 10000 x 101. Save this in a matrix called preds, for example.
5. Use the apply function to compress this to a 1 x 101 matrix (or vector) This is equivalent to having averaged out X (since you have genrated all possible values of X) to get the unconditional distribution of tmax. The code will look something like

```
ypdf <- apply(preds, 2, mean)
```

6. Plot this PDF using plot(density(ypdf)). *Submit* the plot of the PDF.

[Hint:] You can take a look at this workbook (you will need to download the file before viewing it) for an example. The code there is written in keras, but it is doing the exact same thing otherwise.

Answer 1

Getting UQ based on empirical quantiles:

```
knitr::opts_chunk$set(echo = TRUE)
library(cito)
library(torch)
source('BNN_layers.R')
weather <- readRDS("weather.RDS")
library(lubridate)
```

```
Attaching package: 'lubridate'

The following objects are masked from 'package:base':

    date, intersect, setdiff, union
```

```
library(SPQR)
set.seed(303)
```

Original function:

```
bnn_predict0 <- function(model, x_new, n_samples = 100) {

  model$train()  # keep dropout/sampling active

  preds <- torch_stack(lapply(seq_len(n_samples), function(i) {

    model(x_new)$squeeze()

  }), dim = 2)   # shape: (n_test, n_samples)

  return(list(

    mean = preds$mean(dim = 2),

    std  = preds$std(dim = 2)

  ))

}
```

1. Modified function to return a tensor with a dimension n_test x n_samples:

- Modifiy the function to return a tensor with dimensions n_test x n_samples:

```
bnn_predict1 <- function(model, x_new, n_samples = 100) {

  model$train()  # keep dropout/sampling active

  preds <- torch_stack(lapply(seq_len(n_samples), function(i) {

    model(x_new)$squeeze()

  }), dim = 2)   # shape: (n_test, n_samples)

  return(preds) # shape: (n_test, n_samples)

}
```

- Loading data and importing necessary functions:

```
library(torch)
source('BNN_layers.R')
set.seed(303)
```

- Data generation:

```
n <- 300

x_data <- torch_linspace(-3, 3, n)$unsqueeze(2)

y_data <- torch_sin(x_data) + torch_randn(n, 1) * 0.2
```

- Model definition:

```
input_dim <- 1

hidden_dim <- 64

output_dim <- 1

n_epochs <- 500

lr <- 1e-3

# Define model

model <- BayesianNN(input_dim, hidden_dim, output_dim, prior_sigma = 1.0)

control = list(hidden_dim = hidden_dim, n_epochs = n_epochs, lr = lr)

# Fit model

model.fitted <- bnn_fit(X = x_data, Y = y_data, model = model, control =
 ↪  control)
```

```
Training Bayesian Neural Network...
Epoch    ELBO Loss      NLL            KL
-----------------------------------------------------
1        248.2323       211.5133       11015.6836
50       93.8709        57.3028        10970.4072
100      87.2477        50.8016        10933.8281
150      96.0309        59.7054        10897.6426
200      71.8089        35.6002        10862.6025
250      85.8593        49.7689        10827.1143
300      75.4138        39.4368        10793.1045
```

```
350       65.1488         29.2793         10760.8604
400       56.6921         20.9306         10728.4502
450       61.1750         25.5271         10694.3594
500       53.6767         18.1408         10660.7861

Training complete.
```

2. Convert the output to a matrix:

- Predictions with new empirical intervals:

```
x_test <- torch_linspace(-4, 4, 200)$unsqueeze(2)
```

- Geting the predictions

```
result1 <- bnn_predict1(model.fitted, x_test, n_samples = 200)
```

- Convert result to a matrix (each column is a sample)

```
result1 <- as.matrix(result1, nrow = length(x_test), ncol = 200)
```

3. Calculate quantiles for each n_test: - 5% and 95% quantiles for each of the 200 tests

```
result1 <- apply(result1, 1, quantile, c(0.05,0.95))
```

4. Check the proportion of n_test points where these new intervals are wider than the $\mu \pm 2\sigma$ intervals: - Comparing original function intervals to new intervals:

```
# Getting the predictions

result0 <- bnn_predict0(model.fitted, x_test, n_samples = 200)

# Extracting the mean and SD of the variational posterior

mean_pred <- result0$mean

std_pred <- result0$std

result0 <- torch_stack(list(mean_pred, std_pred), dim = 2)

# First column contains the means, second column contains the
# standard deviations
result0 <- as.matrix(result0)

# Lower bound of the intervals
result0 <- cbind(result0, result0[, 1] - 2*result0[, 2])

# Upper bound of the intervals
result0 <- cbind(result0, result0[, 1] + 2*result0[, 2])

# Remove first two columns
result0 <- result0[, -c(1:2)]
```

To compare result0 (original approach) with result1 (empiric approach), we will first transpose result1, then calculate the ranges:

```
result1 <- t(result1)

result1 <- cbind(result1, result1[, 2] - result1[, 1])

result0 <- cbind(result0, result0[, 2] - result0[, 1])
```

We want to check the proportion of the new (empiric) intervals that are wider than the original ones:

```
mean(result1[, 3] > result0[, 3])
```

```
[1] 0
```

We conclude that 13% of the new intervals are wider than the original ones that used the $\mu \pm 2\sigma$ range.

Answer 2

1. Load the weather data and subset July and location 1:

- Load the weather data:

```
weather <- weather[weather$month == 7 & weather$loc == 1,]
```

- Set tmax as Y:

```
Y = weather$tmax
```

- Set pr as X, and convert X to a matrix:

```
X = as.matrix(weather$pr, ncol = 1)
```

2. Fit SPQR model using MLE:

```
model.mle <- SPQR(Y = Y, X = X, method = 'MLE', normalize = T)
```

```
Loss at epoch 1: training: 10.505073, validation: -8.354939
Loss at epoch 10: training: -81.354194, validation: -66.739552
Loss at epoch 20: training: -85.449509, validation: -71.353147
Loss at epoch 30: training: -86.189700, validation: -72.472219
Loss at epoch 40: training: -86.846984, validation: -73.759431
Loss at epoch 50: training: -87.118894, validation: -73.915003
Loss at epoch 60: training: -87.405133, validation: -74.211858
Stopping... Best epoch: 68
Final loss: training: -87.295511, validation: -74.397012
```

3. Simulate precipitation values:

```
# Get the range of precipitation values
precip_range <- range(weather$pr)

# Simulate 10000 precipitation values over the range
precip_sim <- runif(10000, min = precip_range[1], max = precip_range[2])

#precip_range[1]
#precip_range[2]
precip_range
```

```
[1]  0.00000 31.20196
```

```
str(precip_sim)
```

```
 num [1:10000] 12.87 2.39 30.04 18.71 7.26 ...
```

The 'precip_range[1]' and 'precip_range[2]' give us the minimum and maximum precipitation values from the weather dataset, respectively, which are used as the extremes for the uniform distribution in the 'runif' function.
Looking at the range of precipitation values, we can see that they vary from 0 to approximately 32. This means that our simulated precipitation values will also fall within this range, allowing us to explore the relationship between precipitation and maximum temperature (tmax) effectively.

4. Put all the pr values in an X_test matrix and predict the PDF for tmax:

- Put simulated precipitation values in X_test matrix:

```
X_test <- as.matrix(precip_sim, ncol = 1)
```

- Use the predict function to predict the PDF for tmax for each X_test value Use the predict function to predict the PDF for Y for each X. This matrix would have dimensions 10000 x 101. Save this in a matrix called preds, for example.

```
preds <- predict(model.mle, X = X_test, type = 'PDF')
dim(preds) # Check dimensions of preds
```

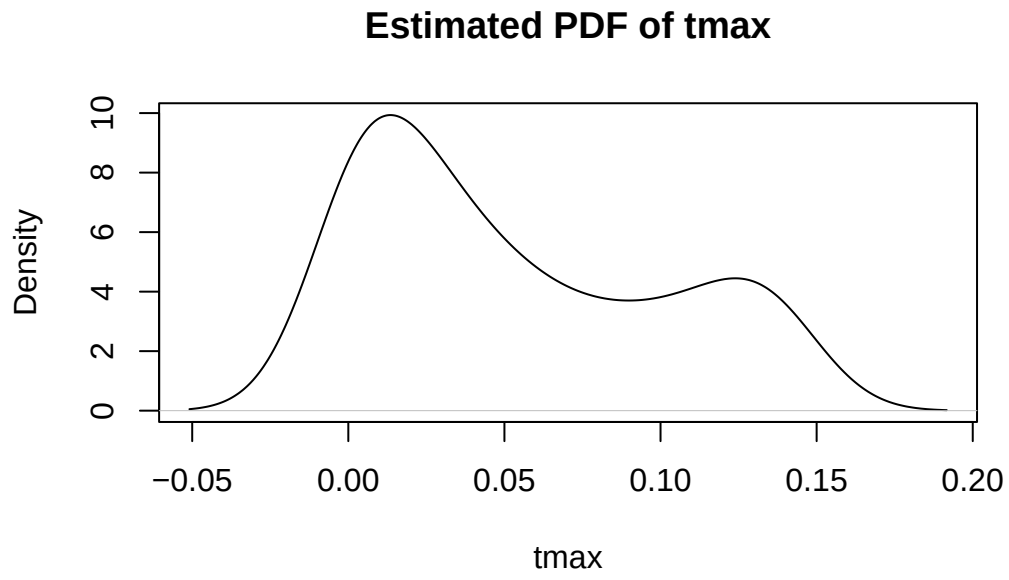
```
[1] 10000   101
```

5. Use the apply function to compress this to a 1 x 101 matrix (or vector) This is equivalent to having averaged out X (since you have generated all possible values of X) to get the unconditional distribution of tmax. The code will look something like:

```
# Average the predictions to get the unconditional distribution of tmax
ypdf <- apply(preds, 2, mean)
```

6. Plot this PDF using plot(density(ypdf)).

```
# Plot the PDF of tmax
plot(density(ypdf), main = "Estimated PDF of tmax", xlab = "tmax", ylab =
 ↪  "Density")
```


Estimated PDF of tmax
Density
0
2
4
6
8
10
−0.05
0.00
0.05
0.10
0.15
0.20
tmax